EVALS · PART 27

Prompts Are Code. Test Them Like It.

Most teams change a prompt, try three examples, and ship. An eval harness is the thing that lets you change it on Friday without fear.

`dataset` `run` `score` `compare` `gate`

16 slides · golden sets · 5 scorer types · pytest for prompts · CI gates · tooling



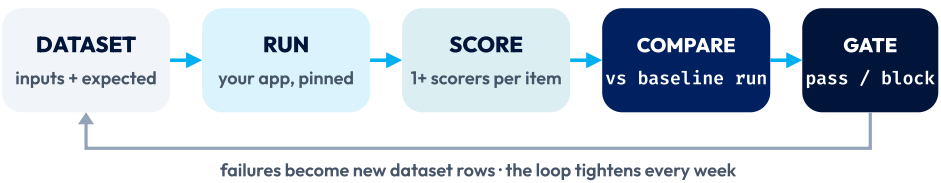
AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

What an eval harness actually is

Five stages in a loop. Every serious LLM team builds this, whether they call it evals, regression tests, or "the benchmark script".



Input: a change (prompt, model, retriever, temperature). Output: a yes/no plus a table of what moved.

Without a harness

"I tried it on five questions and it looks better." Regressions surface as user complaints two weeks later.

With a harness

"Win rate 58% over baseline on 400 items, refunds slice unchanged, cost −12%, p95 latency +40 ms." Merge.

The harness is not a metric. It is the *infrastructure* that lets any metric run on every change, automatically, with a diff.

Build the golden dataset

The dataset is 80% of the value. Four sources, mixed on purpose, versioned like code.

Real logs · ≈50%

Sample production requests, scrub PII, label the good ones. This is what users actually ask, spelling mistakes included.

Edge cases · ≈20%

Empty input, 10K-token input, two questions in one, wrong language, table pasted as text, question with no answer in the docs.

Adversarial · ≈15%

Prompt injection, requests for competitor info, jailbreak attempts, off-policy asks. Expected output is often a refusal.

Past failures · ≈15%

Every bug report becomes a row. This is the part that grows forever and stops the same regression twice.

ONE ROW · JSONL

```
{"id": "refund-017", "slice": "refunds", "lang": "en",
 "input": "bought it 20 days ago, can i still return?",
 "expected": {"must_mention": ["14-day"], "must_not": ["30-day"], "refer
```

50–100
rows to start, per feature

300+
before win rates are stable

git
or a versioned store. Never a shared sheet.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

Five scorer types, cheapest first

Every row runs through several scorers. Deterministic ones run on all rows for free. Expensive ones run where the cheap ones cannot decide.

SCORER	EXAMPLE	COST	CATCHES
Exact / regex	must_mention "14-day", no phone numbers	free	Policy facts, banned content, format
Schema	Pydantic model validates the JSON	free	Broken tool calls, missing fields
Embedding sim	cosine(answer, reference) ≥ 0.85	≈$0	Paraphrased correct answers
LLM judge	rubric: faithful to context? 1–5	≈$0.005	Reasoning quality, hallucination
Human spot-check	20 random + 20 lowest-scored per run	minutes	Everything the judge missed



Budget shape for a 400-row run. Deterministic scorers fail fast and skip the judge call entirely.
Store every scorer's result per row. "Passed schema, failed judge" is a different bug from "failed schema".

</> If the task output can be executed (code, SQL, a tool call against a sandbox), execution **is** the scorer. Nothing else comes close.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

Pytest for prompts, parametrized

You already have a test runner, a CI, and a habit. Reuse all three. One JSONL file becomes hundreds of test cases.

```python
# tests/test_support_bot.py
import json, pytest
from app import answer            # the function under test
from scorers import contains_all, contains_none, judge_faithful

ROWS = [json.loads(l) for l in open("evals/support.jsonl")]

@pytest.mark.parametrize("row", ROWS, ids=[r["id"] for r in ROWS])
def test_row(row, record_property):
    out = answer(row["input"])
    record_property("tokens", out.usage.total_tokens)
    record_property("latency_ms", out.latency_ms)
    exp = row["expected"]
    assert contains_all(out.text, exp.get("must_mention", []))
    assert contains_none(out.text, exp.get("must_not", []))
    if "reference" in exp:
        score = judge_faithful(row["input"], out.text, exp["reference"])
        record_property("judge", score)
        assert score >= 3, out.text[:200]
```

`pytest -n 8 -k refunds` runs one slice in parallel. Cache LLM calls by input hash so reruns are free.

`record_property` lands in JUnit XML. Your CI already renders it per test, per run.

Run it on every change that matters

Code diffs are not the only thing that changes model behaviour. Most LLM regressions come from changes that never touch your repo.

CHANGE	VISIBLE IN GIT?	RUN
Prompt text	yes	Full suite on the PR
Model ID or version	yes, if pinned	Full suite + cost and latency diff
Retriever / index rebuild	no	RAG slices, nightly
Provider silent update	no	Smoke suite daily against prod config
Temperature, max_tokens	yes	Full suite. Small knobs, large effects
Tool schema / description	yes	Agent slices. Descriptions steer tool choice



⚠ **Unpinned model aliases like "latest" mean your nightly can fail with zero commits. That is the point of the nightly.**

Compare runs, not scores

A single number hides everything. The useful output of a harness is a diff table: candidate vs baseline, per slice, with cost and latency beside quality.

RUN #142 (NEW PROMPT) VS #138 (MAIN) · 400 ROWS

SLICE	N	WIN / TIE / LOSS	JUDGE Δ	COST Δ	P95 Δ
all	400	46 / 38 / 16	+0.21	-9%	+35 ms
faq	180	52 / 40 / 8	+0.30	-12%	+20 ms
refunds	90	30 / 30 / 40	-0.35	-4%	+30 ms
hindi	70	44 / 40 / 16	+0.12	-8%	+90 ms
adversarial	60	50 / 45 / 5	+0.10	-11%	+15 ms

What the aggregate hid

Overall +0.21 looks like a win. Refunds lost 40% of pairs. The new prompt dropped the returns-policy paragraph. Aggregate said ship, slice said stop.

Always report four things

Quality (win rate or judge mean), cost per request, p50/p95 latency, and failure count by scorer. A quality win that doubles cost is a decision, not a merge.

Win rates on < 100 rows swing ±10 points from noise alone. Show a confidence interval or do not show a decimal.

Thresholds and flaky handling

LLM outputs are non-deterministic even at temperature 0. A gate that fails on one bad sample will be ignored within a week.

Gate rules that hold up

- Deterministic scorers: **0 new failures**, hard block
- Pairwise vs main: win rate ≥ 45% (no regression)
- No slice drops > 5 pts judge mean
- Cost per request ≤ +20% without a label
- p95 latency ≤ +25%

Taming flakes

- Run each judged row **3×**, take the median
- Cache by (model, prompt, input) hash: reruns cost 0
- Quarantine known-flaky rows with a ticket, not deletion
- Retry only on transport errors, never on a bad score

WHY MEDIAN OF 3

Same row, same prompt, temperature 0, three runs:



Cost is 3× the judge calls on that row. Cheaper than a team that has learned to click "re-run" without reading.

⚠ Do not gate on an absolute judge score like "mean ≥ 4.0". Judge scales drift. Gate on the **diff against the baseline run** executed with the same judge, same day.

From a red row to the exact request

A failing eval that only says "score 2" is a dead end. Every result row must link to the full trace of what the app did for that input.

`refund-017 · judge 2/5 · "mentions 30-day window"` click ↓

retrieval	prompt	model call	output
5 chunks · 180 ms policy_v1.md (stale)	2,140 tokens template v14 · hash a3f9	gpt-4.1-mini · t=0 1.4 s · $0.0031	312 tokens "...within 30 days..."

Root cause in 30 seconds

The prompt was fine. Retrieval returned a stale policy file. The fix is an index rebuild, not a prompt edit. Without the trace you would have edited the prompt.

Minimum to store per row

Prompt hash, model ID, all retrieved chunk IDs, raw output, token counts, latency, and a trace ID that opens in Langfuse, LangSmith or your OTel backend.

Rule: an eval failure without a trace link is a bug in the harness, not in the app.

A harness in 40 lines

No framework. Dataset in, results table out, exit code for CI. Everything else is an upgrade of this.

```
# eval.py  ·  python eval.py --baseline runs/main.json
import json, sys, hashlib, statistics as st
from app import answer
from scorers import deterministic, judge

rows = [json.loads(l) for l in open("evals/support.jsonl")]
results = []
for r in rows:
    out = answer(r["input"])                        # cached by hash insi
    det = deterministic(out.text, r["expected"])    # dict of bools
    j = st.median(judge(r, out.text) for _ in range(3)) if all(det.value
    results.append({**r, "output": out.text, "det": det, "judge": j,
                    "tokens": out.tokens, "ms": out.ms, "trace": out.tra

run_id = hashlib.sha1(open("prompts/support.txt","rb").read()).hexdigest
json.dump(results, open(f"runs/{run_id}.json", "w"))
base = json.load(open(sys.argv[2]))
for slice_ in sorted({r["slice"] for r in rows}):
    cur = [x["judge"] for x in results if x["slice"] == slice_]
    old = [x["judge"] for x in base if x["slice"] == slice_]
    print(f"{slice_:12} n={len(cur):3} judge {st.mean(cur):.2f} (Δ {st.m
hard_fail = sum(not all(x["det"].values()) for x in results)
sys.exit(1 if hard_fail else 0)                     # CI reads this
```

When to stop writing your own

The script gets you to 200 rows. After that you want run history, a UI for humans, and scorers you did not write.

TOOL	SHAPE	STRONG AT
promptfoo	YAML + CLI, open source	Matrix of prompts × models, red-team plugins, CI-native
DeepEval	pytest plugin, open source	Built-in metrics (faithfulness, G-Eval), feels like unit tests
Ragas	Python library, open source	RAG-specific: context precision/recall, answer faithfulness
Inspect	Python framework (UK AISI)	Agentic and multi-step evals, sandboxed tool use
Langfuse	Open-source platform	Tracing + datasets + scores in one place, self-hostable
LangSmith	Hosted platform	Tracing, dataset UI, annotation queues, LangChain-native
Braintrust	Hosted platform	Run comparison UI, playground, human review workflows
OpenAI Evals	Registry + API	Model-graded evals inside the OpenAI dashboard

✓ **Pick on three things: run history with a diff view, a labelling UI a non-engineer can use, and a trace link from every failed row. Keep the dataset as JSONL in git whatever you choose. Tools change, your 2,000 labelled rows are the asset.**

ANTI-PATTERNS

Six ways evals lie to you

Each of these produced a green CI and a worse product.

01 **Evaluating on your few-shot examples.** The prompt contains the answers. 100% pass, zero information. Keep eval rows and prompt examples disjoint.

02 **One number.** "Score went from 3.8 to 4.1." Which slice? At what cost? With what variance? A single metric is a vibe with decimals.

03 **No slices.** 70% of rows are easy FAQs. The 10% that are refunds regress silently inside a rising average.

04 **Overfitting to the judge.** Prompt tuned until the judge is happy, humans see no change. Keep a hold-out set the tuning loop never touches.

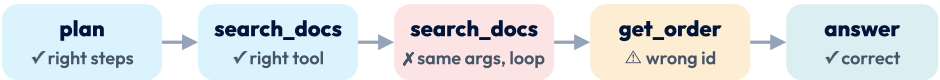
05 **Stale golden set.** Product added a new plan tier, dataset still tests the old one. Every user-facing change adds rows the same week.

06 **Evals only at PR time.** Provider updated the model on Tuesday. No commits, no eval, users noticed Friday. Nightly canary against prod config.

A harness that has never blocked a merge is not measuring anything. Check how often it fails, and whether anyone reads why.

Evaluating multi-step agents

A final answer can be right by luck after six wrong tool calls. Agent evals score the trajectory, not just the last message.



Outcome score: 5/5. Trajectory score: 2/5. Cost: 4 tool calls where 2 would do. Latency: 9 s.
Ship this and your bill doubles while the dashboard stays green.

LEVEL	SCORER	EXAMPLE ASSERTION
Outcome	reference or judge	Final answer matches expected order status
Trajectory	deterministic on the trace	Tool sequence ⊆ allowed set, no repeated (tool, args), ≤ 4 steps
Tool args	schema + exact	get_order called with the id from the user message
Safety	deterministic	No write tool called before confirmation step

Run agent evals against **mocked or sandboxed tools** with fixed responses. Otherwise the eval tests your database's mood, not the agent.

GETTING STARTED

Your first harness in five days

You do not need a platform, a budget or a data team. You need one JSONL file and a script that fails.

Day 1 **Pull 50 real requests** from logs. Scrub PII. Write the expected behaviour for each as must_mention / must_not / reference. Commit to `evals/`.

Day 2 **Deterministic scorers only.** The 40-line script. Run it on main. Fix the 3 rows that fail. You now have a baseline.

Day 3 **Add 20 edge cases and 10 adversarial rows.** Tag every row with a slice. Print results per slice.

Day 4 **Add one LLM judge scorer** for the rows with a reference. Median of 3. Wire the exit code into CI as a required check.

Day 5 **Make a change on purpose.** Swap the model or cut a paragraph. Watch the diff table catch it. Show the team. Now every bug report adds a row.

80	≈ $2	1
rows by Friday	judge cost per full run	regression caught pays for the week

Eight rules for evals that hold up

These separate a harness people trust from a script people bypass.

1. Dataset is **versioned JSONL in git**. Every row has a slice and a source.
2. Deterministic scorers run on **every row**. Judges run on the survivors.
3. Compare to a **baseline run**, never to an absolute score.
4. Report quality, **cost, latency and failures** together, per slice.
5. Judged rows run **3×, median**. Cache by input hash.
6. Every failing row links to a **full trace**.
7. PR smoke, merge full, **nightly canary** on prod config.
8. Every bug report and every product change **adds rows** that week.

NUMBERS TO REMEMBER

300	±10	45%	5 pts
rows before win rates stabilise	pts of noise under 100 rows	pairwise floor to not regress	max slice drop before block

Eval harness on one slide

CONCEPT	REMEMBER
The loop	Dataset → run → score → compare → gate. Failures feed back as rows.
Golden set	50% real logs, 20% edge, 15% adversarial, 15% past bugs. JSONL in git, sliced.
Scorers	Regex → schema → embedding → LLM judge → human. Cheapest first, all logged.
Execution	If output can run (code, SQL, tools), running it is the scorer.
Pytest	parametrize over rows, record_property for tokens and latency, -n for parallel.
Triggers	PR smoke (50 rows), merge full (400), nightly canary on prod config.
Compare	Diff vs baseline run, per slice: win/tie/loss, judge Δ, cost Δ, p95 Δ.
Gates	0 deterministic fails, win rate ≥ 45%, no slice −5, cost ≤ +20%.
Flakes	Median of 3 judge runs, cache by hash, quarantine with a ticket.
Traces	Every row stores prompt hash, model, chunk IDs, tokens, trace ID.
Agents	Score trajectory and tool args, not just the final answer. Mock the tools.
Tools	promptfoo, DeepEval, Ragas, Inspect, Langfuse, LangSmith, Braintrust.

A prompt without an eval is a guess with a deploy button. Build the loop, then change anything you like.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost